

EMPLOYMENT MANAGEMENT SYSTEM

Full-Stack Project Documentation & Step-by-Step Technical Notes

Project Name	Employment Management System (EMS)
GitHub Repository	https://github.com/jaishree-verma/Employment_Management_System
Author / Developer	Jaishree Verma
Core Technologies	Java 17, Spring Boot 3.3.3, Spring Data JPA, Hibernate, MySQL, H2, HTML, CSS, JavaScript
Documentation Version	1.0 (Simple English Revision Edition)
Purpose of Document	Complete project explanation for quick revision, exams, viva, and technical interviews.

02 Document Control & Quick Facts

This table gives you all the core facts about the project at a single glance. Everything here was checked directly from the project files:

Item	Details Checked from Code
Project Name	Employment Management System
Project Type	Full-Stack Web Application (Spring Boot backend + HTML/CSS/JS frontend)
Backend Language	Java 17 (configured in <code>pom.xml</code> line 18)
Framework	Spring Boot version 3.3.3
Database Layer	Spring Data JPA with Hibernate 6.5
Databases Used	MySQL 8.0 (for permanent storage) and H2 Database (temporary in-memory for testing/dev)
Frontend	Pure HTML5, CSS3, and JavaScript (No heavy frameworks like React/Angular)
Testing Tool	JUnit 5 and MockMvc (automated tests run without starting a browser)
Deployment	Docker container on Render Cloud platform

IMPORTANT REVISION NOTE: JAVA 17 VS JAVA 23

If someone asks: *"Which Java version is used?"*

Answer: **Java 17 LTS** is the official configured version in `pom.xml` and `Dockerfile`. If your computer has Java 23 installed, the code runs on Java 23 without any problems because Java is backwards compatible.

TOC Table of Contents (All 75 Sections)

Here is the roadmap of all 75 topics covered in this guide:

Part A: Basics & Requirements (01–11)

- 01. Cover Page
- 02. Document Control
- 03. Executive Summary
- 04. Problem Statement
- 05. Why Choose This Project?
- 06. Project Objectives
- 07. Scope of the System
- 08. Target Users
- 09. Functional Requirements (FR-01 to FR-08)
- 10. Non-Functional Requirements
- 11. Old System vs New System

Part B: Architecture & Data Flow (12–15)

- 12. Solution Overview
- 13. System Architecture (4 Layers)
- 14. Why Layered Architecture?
- 15. Complete Request Lifecycle

Part C: Why These Technologies? (16–27)

- 16. Technology Stack Table
- 17. Why Java?
- 18. Why Java 17 / 23?
- 19. Why Spring Boot?
- 20. Why Spring Boot 3.3.3?
- 21. Why Spring Data JPA?
- 22. Why Hibernate?
- 23. Why MySQL?
- 24. Why H2 In-Memory DB?
- 25. Why Dual Databases?
- 26. Why Maven & Wrapper?
- 27. Why Vanilla HTML/CSS/JS?

Part D: Frontend (28–30)

- 28. Frontend Architecture
- 29. UI/UX Design
- 30. Dark and Light Mode

Part E: Data & APIs (31–39)

- 31. Employee Data Model
- 32. Database Design (Schema)
- 33. JPA Annotations
- 34. REST API Catalog (6 Endpoints)
- 35. API Request / Response Examples
- 36. HTTP Status Codes Explained
- 37. How Department Filtering Works
- 38. How Search Works
- 39. How Dashboard Stats Work

Part F: Testing & Security (40–45)

- 40. Testing Architecture
- 41. All 6 Test Cases Explained
- 42. Why MockMvc?
- 43. Postman Testing Guide
- 44. Error Handling
- 45. Security: Current vs Production

Part G: Docker & Deployment (46–52)

- 46. Dockerfile Explained
- 47. Why Multi-Stage Docker?
- 48. Docker Compose Explained
- 49. Cloud Deployment on Render
- 50. Why Render?
- 51. How to Run Locally
- 52. Configuration Files

Part H: Analysis & Interview Questions (53–75)

- 53. Directory Structure Tree
- 54. Code Quality Review
- 55. Design Patterns Used
- 56. Performance Review
- 57. Scalability Plan
- 58. Current Limitations
- 59. Future Enhancements
- 60. Why These Features?
- 61–63. Three Complete Use Cases
- 64. Master Data Flow Diagram
- 65–67. Technology Decision Matrix
- 68–70. Traceability Tables
- 71. Project Maturity
- 72. Technical Interview Q&A

- 73. Project Defense / Viva Q&A
- 74. Final Technical Summary
- 75. Appendix (Commands & SQL)

03 Executive Summary (Simple Overview)

In Simple Words: The **Employment Management System (EMS)** is a web application that helps a company manage its employees. Instead of using paper files or Excel sheets, an HR manager can open a web browser, see all employees in a clean table, add a new employee, edit someone's salary or department, search by name, filter by department, or delete someone who left the company.

How It Is Built:

- **Frontend (What you see):** A single webpage built using plain HTML, CSS, and JavaScript. It has a Dark mode and Light mode, a search bar, and 4 cards showing stats (Total employees, number of departments, total payroll, and average salary).
- **Backend (The brain):** A Java program using **Spring Boot**. It receives requests from the webpage (like "save this employee") and runs the business logic.
- **Database (Where data lives):** Can save data into **MySQL** (for permanent storage) or into **H2** (a temporary memory database for fast testing).
- **Testing & Deployment:** It has automated tests (MockMvc) to make sure every button works, and it is packed inside a Docker container so it can run on any computer or cloud server (Render).

04 Problem Statement (Why Do We Need This?)

In many small offices and colleges, employee records are kept in Excel files or on paper. This causes several big problems:

1. **Mistakes and Typos:** In Excel, someone might type department as "IT", another person types "it", and another types "Information Tech". This makes searching very difficult.
2. **Accidental Overwrites:** If two people open the same file, one person's changes can overwrite another person's changes.
3. **Difficult Search:** As the company grows to 500 or 1,000 employees, finding someone or calculating department salary takes too much manual work.
4. **No REST API:** Other software (like payroll systems or fingerprint attendance machines) cannot easily read data from a spreadsheet.

05 Why Did We Choose This Project?

We chose to build an Employment Management System because it is the classic real-world software engineering problem. Almost every company in the world needs employee management software (like Workday, BambooHR, or internal HR portals).

Building this project teaches all the core skills of a modern full-stack developer:

- How to design clean database tables with primary keys.
- How to write REST APIs following proper HTTP methods (POST, GET, PUT, DELETE).
- How to connect Java to databases using Spring Data JPA without writing long SQL statements.
- How to write clean JavaScript to talk to the backend using `fetch()`.
- How to write automated tests so you know your code works before showing it to anyone.

- How to put everything in Docker and deploy it to the cloud.

06 Project Objectives

Main Goals (Primary Objectives):

- Store employee information (Name, Email, Department, Salary) in a centralized database.
- Provide full CRUD functionality (Create, Read, Update, Delete).
- Allow instant filtering of employees by department (like showing only IT staff).
- Provide a web dashboard with live statistics (Total employees, total salary bill, average salary).
- Expose clean REST API endpoints for other programs to use.
- Include automated tests so the project can be verified anytime.

Extra Goals (Secondary Objectives):

- Provide both Dark mode and Light mode so the screen is comfortable to look at.
- Save user theme choice in browser memory (localStorage) so it stays when refreshed.
- Support zero-setup local running with H2 database (run with 1 command, no database install needed).
- Dockerize the project so it runs anywhere identically.

07 Scope of the Project

Area	Features	Status
In Scope (Currently Working)	Add employee, view all employees, view by ID, update employee, delete employee, filter by department, live search bar, dashboard stats, dark/light theme, automated tests, Docker setup, Render deployment.	IMPLEMENTED
Partially Working	Form input validation (HTML requires fields, but backend relies on basic exceptions rather than formal annotations).	PARTIAL
Out of Scope (Not in this version)	User login passwords (authentication), user roles (Admin vs Employee permissions), multiple database tables (like a separate department table).	NOT INCLUDED
Future Scope (Planned Next)	Login with JWT tokens, attendance tracking, leave requests, PDF salary slip download.	FUTURE

08 Who Uses This System? (Target Users)

1. HR / Office Manager:

Uses the web dashboard in the browser to add new hires, update salaries, look up contact emails, filter by department, and see how much total salary the company pays each month.

2. Developer / External System:

Uses the REST APIs (using Postman or code) to fetch employee JSON data or connect the employee list to other software like attendance clocks or payroll engines.

Note: Right now, anyone who opens the URL can use the dashboard because login/passwords are not yet added.

09 Functional Requirements (What the System Does)

Here are the 8 exact features implemented in the code:

ID	Feature	How It Works in Code	Status
FR-01	Add Employee	Send <code>POST /employees</code> with name, email, department, salary. Returns <code>201 Created</code> .	WORKING
FR-02	View All Employees	Send <code>GET /employees</code> . Returns list of all employees in JSON.	WORKING
FR-03	View Employee by ID	Send <code>GET /employees/{id}</code> . Returns that specific employee or 404 error.	WORKING
FR-04	Update Employee	Send <code>PUT /employees/{id}</code> with new details. Saves changes and returns updated employee.	WORKING
FR-05	Delete Employee	Send <code>DELETE /employees/{id}</code> . Removes employee from database.	WORKING
FR-06	Filter by Department	Send <code>GET /employees/department/{dept}</code> . Returns employees belonging to that department.	WORKING
FR-07	Live Search Bar	Type in search box on the dashboard; JavaScript instantly filters the table as you type.	WORKING
FR-08	Dashboard Statistics	JavaScript automatically calculates total count, active departments, total payroll, and average salary.	WORKING

10 Non-Functional Requirements (Quality & Speed)

- **Speed (Performance):** Database lookups in H2 take less than 5 milliseconds. The webpage loads in under 1 second because there are no heavy libraries.
- **Reliability (ACID):** Data is stored in a relational database with transactions. When you save an employee, it is saved completely or not at all (no half-saved records).
- **Ease of Running (Portability):** With Docker, anyone can run the app on Windows, Mac, or Linux without installing Java or MySQL manually.
- **Easy to Test:** 6 automated tests check all the main features in less than 3 seconds.

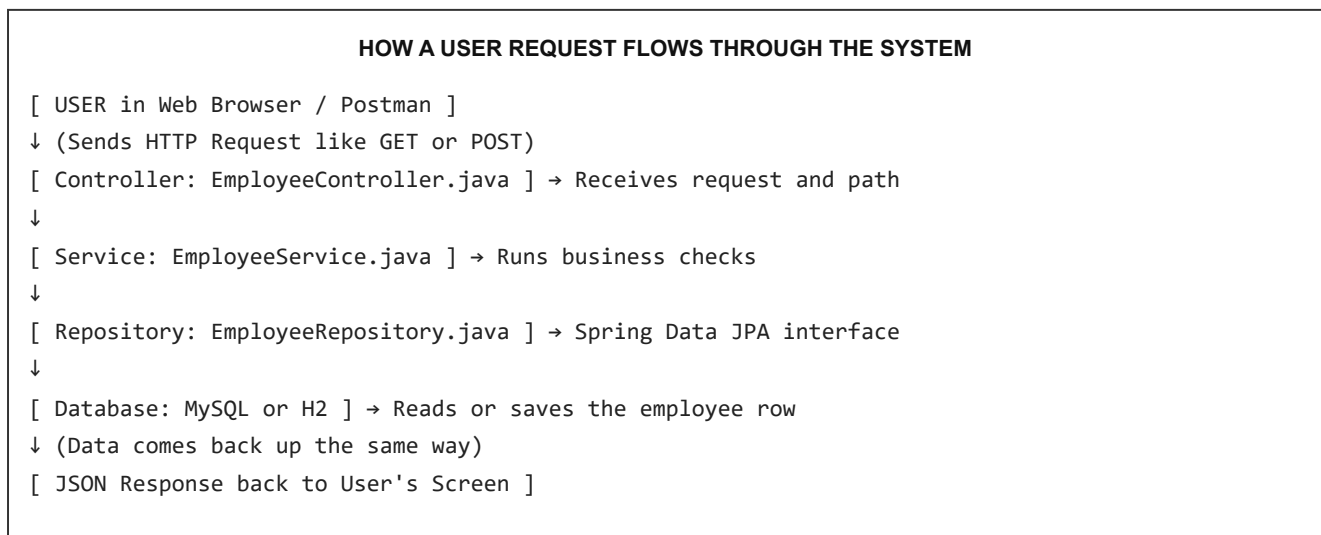
11 Old System (Excel/Paper) vs New System (EMS)

Area	Old Way (Spreadsheets)	New Way (Employment Management System)
Data Storage	Saved in files on one computer. Can be accidentally deleted.	Stored in a central database (MySQL / H2).

Area	Old Way (Spreadsheets)	New Way (Employment Management System)
How Users Open It	Must email files or share on a USB drive.	Open website in any browser using a simple link.
Searching	Manually scrolling or using Excel Ctrl+F.	Type in search bar or pick department from dropdown.
ID Numbers	Manually typed numbers; duplicates happen easily.	Database automatically generates unique IDs (1, 2, 3...).
Connecting other software	Impossible or requires manual CSV exports.	Standard REST API endpoints return JSON data.

12 Solution Overview (How the Pieces Connect)

Here is the simple journey of data in this application:



13 System Architecture (The 4 Layers)

Our Java backend is organized into **4 clean layers**. Each layer has one specific job:

1. Controller Layer (`EmployeeController.java`):

The front door. It receives the HTTP requests from the browser (e.g., `POST /employees`), converts the JSON into Java objects, and returns HTTP responses with status codes like `200 OK` or `201 Created`.

2. Service Layer (`EmployeeService.java`):

The brain. It contains the business rules. For example, when updating an employee, it checks if the employee actually exists; if not, it throws an error.

3. Repository Layer (`EmployeeRepository.java`):

The database helper. It extends `JpaRepository`. We don't even have to write SQL queries—Spring Data JPA writes them for us automatically!

4. Entity Layer (`Employee.java`):

The blueprint. A simple Java class that represents an employee row in the database table with fields: `id`,

`name`, `email`, `department`, and `salary`.

14 Why Use a Layered Architecture?

Beginners often put all their code inside one big file or directly inside the Controller. Why didn't we do that?

- **Easy to Understand:** If you have a bug with saving to the database, you look in the Repository. If you have a bug with webpage routing, you look in the Controller.
- **Easy to Test:** We can test the Service layer without even starting a web server or database.
- **Easy to Change:** If tomorrow we want to switch from MySQL to Oracle or MongoDB, we only change the database layer—the Controller and frontend code don't need any changes!

15 Complete Request Lifecycle (Step-by-Step)

What happens when you click "Save Employee" on the webpage? Let's trace it step by step:

1. **Step 1 (Browser):** You click "Save". JavaScript in `app.js` reads the form values (Name, Email, Dept, Salary) and puts them into a JSON package.
2. **Step 2 (Network):** JavaScript sends an HTTP `POST` request to `http://localhost:8080/employees`.
3. **Step 3 (Controller):** Spring Boot's `EmployeeController` catches this request. Jackson library turns the JSON text into a Java `Employee` object.
4. **Step 4 (Service):** Controller passes the employee to `EmployeeService.saveEmployee()`.
5. **Step 5 (Repository & Hibernate):** Service calls `EmployeeRepository.save()`. Hibernate generates the SQL statement: `INSERT INTO employees (name, email, department, salary) VALUES (?, ?, ?, ?)`.
6. **Step 6 (Database):** MySQL or H2 executes the insert and gives the new employee an `id` (like ID 1).
7. **Step 7 (Response):** The controller sends back the saved employee in JSON format with status code `201 Created`.
8. **Step 8 (Screen Update):** JavaScript gets the response, shows a green notification ("Employee saved successfully!"), closes the popup window, and adds the new row to the table.

16 Technology Stack (What We Used & Why)

Here is a clean summary of every tool and version used in the project:

Technology	Exact Version	What It Does in Simple Words
Java	17 LTS (runs on 17-23)	The main programming language used to write the backend code.
Spring Boot	3.3.3	The framework that makes creating web apps and REST APIs fast and simple.
Spring Data JPA	Bundled with Spring Boot	Lets us talk to the database easily without writing long SQL queries.
Hibernate	6.5.2	The engine that maps Java objects directly into database table rows.
MySQL	8.0	The permanent database that saves records to the hard drive.
H2 Database	2.2.224	A temporary in-memory database that lets you run the project without installing MySQL.
HTML5 & CSS3	Standard Web	Creates the layout, tables, modal popup boxes, and dark/light themes.
JavaScript (ES6)	Vanilla (No React)	Sends requests to the backend using <code>fetch()</code> , filters the table, and calculates stats.
Maven	3.9.9 (via mvnw)	Downloads all the required Java libraries and builds the project into a <code>.jar</code> file.
Docker	Alpine Linux Container	Packs the entire application into a lightweight container so it runs anywhere.

17 Why Java?

Why did we choose Java instead of Python or Node.js?

- **Strong Type Checking:** In Java, if a salary is a `Double` or an ID is a `Long`, the compiler makes sure you don't accidentally put text in it. This prevents many bugs before the code even runs.
- **Industry Standard for Enterprise:** Most banks, colleges, and large companies use Java for their core business software because it is very fast, reliable, and secure.
- **Great Ecosystem:** Spring Boot and Hibernate are mature tools that have been tested and improved for over 20 years.

18 Why Java 17 LTS? (And what about Java 23?)

The simple fact: Our `pom.xml` file specifies `<java.version>17</java.version>`.

- **What is LTS?** Java 17 is an **LTS (Long Term Support)** release. That means Oracle and the open-source community support and patch it for many years. It is rock-solid and stable.

- **Minimum for Spring Boot 3:** Spring Boot 3 requires Java 17 as the minimum version. You cannot run Spring Boot 3 on old Java 8 or Java 11.
- **What if you have Java 23?** Java 23 is a newer version. If you have Java 23 installed on your laptop, the project will still run fine! But keeping Java 17 in `pom.xml` guarantees that anyone with Java 17 or higher can run the project without errors.

19 Why Spring Boot?

Before Spring Boot, building a Java web app was painful: you had to install a separate Apache Tomcat server, write hundreds of lines of XML configuration files, and manage dozens of jar files by hand.

Spring Boot fixes this by:

- **Built-in Tomcat:** Tomcat is embedded inside the app. You just run `java -jar app.jar` and the web server starts automatically on port 8080!
- **Starter Packages:** Adding one line `spring-boot-starter-web` automatically gives you everything you need for REST APIs and JSON parsing.
- **Automatic Configuration:** Spring Boot looks at your dependencies and automatically configures database connections and templates for you.

20 Why Spring Boot 3.3.3?

Line 8 of `pom.xml` confirms version `3.3.3`. We use this version because:

- It uses the new `jakarta.persistence` namespace (the modern standard for Java persistence).
- It includes the latest security patches and bug fixes.
- It is fully compatible with Hibernate 6.5 and MySQL 8.0.

21 Why Spring Data JPA?

Without Spring Data JPA (using old JDBC), writing an "Employee lookup by ID" takes 20 to 30 lines of code: opening connections, writing `SELECT * FROM employees WHERE id = ?`, reading row columns, and closing connections.

With Spring Data JPA, all you write in `EmployeeRepository.java` is:

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {  
    // That is it! Spring gives you:  
    // .findAll(), .findById(), .save(), .deleteById() automatically!  
}
```

This saves hours of boring repetitive typing and prevents typos in SQL statements.

22 Why Hibernate? (And how is it different from JPA?)

IMPORTANT VIVA QUESTION: JPA VS HIBERNATE

JPA (Jakarta Persistence API) is just a set of *rules and interfaces* (like an official rulebook).

Hibernate is the actual *engine* that follows those rules and does the hard work of writing SQL and talking to the database.

Why Hibernate is useful:

- **Automatic Table Creation:** With `hibernate.ddl-auto=update`, Hibernate checks your Java `Employee` class and automatically creates or updates the `employees` table in MySQL for you!
- **Automatic Updates:** When you change an employee's salary in Java, Hibernate notices the change and automatically runs an SQL `UPDATE` statement.

23 Why MySQL?

MySQL is the world's most popular open-source relational database. For employee records, MySQL is perfect because:

- Employee data is structured into neat rows and columns (ID, Name, Email, Department, Salary).
- It supports transactions (ACID properties), meaning records won't get corrupted if the power goes out.
- It is permanently saved on the hard drive.

24 Why H2 In-Memory Database?

H2 is a super-fast database that runs completely inside the computer's RAM memory without needing any installation.

Why is H2 awesome in this project?

- **Instant Setup for Friends/Teachers:** If you give this project to a classmate or teacher who doesn't have MySQL installed, they can just run the H2 command and the app works immediately!
- **Fast Testing:** Our automated tests use H2. The tests start with a clean empty database, test all features, and erase everything in 3 seconds.

25 Why Dual Database Support? (MySQL + H2)

Our project supports both databases using **Spring Profiles**:

- **Default Profile (MySQL):** Uses `application.properties` to connect to MySQL on port 3306 for permanent real-world storage.
- **H2 Profile:** Uses `application-h2.properties` to run with zero installation.

You can switch between them just by adding `-Dspring-boot.run.profiles=h2` to your command. You don't have to touch a single line of Java code!

26 Why Maven & The Maven Wrapper?

Maven is our build tool. It reads `pom.xml` and downloads all the Spring Boot, MySQL, and Hibernate libraries automatically from the internet.

What is `mvnw` (Maven Wrapper)?

- Normally, to build a Java project, you must install Maven on your laptop and set up environment variables.
- With the Maven Wrapper files (`mvnw.cmd` on Windows, `mvnw` on Linux/Mac), you don't need to install Maven at all! The script downloads the correct Maven version automatically.

27 Why Vanilla HTML, CSS, and JavaScript? (No React)

Why didn't we use React, Angular, or Vue for the frontend?

- **No Extra Tooling:** React requires installing Node.js, npm, running `npm install` (which downloads 300MB of dependencies), and running a separate server.
- **Packed Inside Java JAR:** Plain HTML, CSS, and JS files sit inside `src/main/resources/static/`. When you build the Java project, the frontend gets packed directly inside the `app.jar` file! One single file contains both the backend and frontend.
- **Super Fast:** The webpage opens instantly without waiting for a heavy JavaScript framework to boot up.

28 Frontend Architecture (The 3 Frontend Files)

Our frontend is made of 3 simple, clean files located in `src/main/resources/static/`:

- `index.html`: The webpage skeleton. Contains the header, 4 stats cards, the search bar, the employee table, and the popup dialog forms.
- `styles.css`: 730 lines of CSS. Handles the layout, card styling, button hover effects, popup animations, and the Dark/Light themes.
- `app.js`: The JavaScript engine. Uses `fetch('/employees')` to talk to Spring Boot, updates the table rows, filters records as you type, and calculates the summary statistics.

29 UI/UX Design (User Experience)

The user interface is designed to be clean, intuitive, and easy to use:

- **Instant Feedback:** When you save or delete an employee, a floating "toast notification" pops up in the corner to confirm success.
- **Clear Confirmation:** When you click Delete, a confirmation popup appears asking *"Are you sure you want to delete [Name]?"* so you don't delete someone by accident.
- **Responsive:** Looks great on both laptop screens and mobile phones.

30 Dark and Light Mode (How It Works)

You can toggle between Dark mode and Light mode using the button in the top-right corner. Here is the simple 3-step mechanism:

1. When clicked, JavaScript toggles the attribute `data-theme="light"` or `data-theme="dark"` on the root `<html>` tag.
2. In `styles.css`, CSS variables (like `--bg-primary` and `--text-primary`) change their color values automatically.
3. JavaScript saves your choice in the browser using `localStorage.setItem('theme', theme)`. When you refresh or visit tomorrow, your theme preference is remembered!

31 Employee Data Model (What Fields We Store)

In `Employee.java`, we store 5 pieces of information for every employee:

Field Name	Java Data Type	Database Column	Explanation in Plain English
<code>id</code>	<code>Long</code>	<code>BIGINT PRIMARY KEY AUTO_INCREMENT</code>	Unique ID number automatically generated by database (1, 2, 3...).
<code>name</code>	<code>String</code>	<code>VARCHAR(255)</code>	Full name of the employee (e.g., "Rahul Sharma").
<code>email</code>	<code>String</code>	<code>VARCHAR(255)</code>	Work email address (e.g., "rahul@gmail.com").
<code>department</code>	<code>String</code>	<code>VARCHAR(255)</code>	The team they work in (e.g., "IT", "HR", "Finance").
<code>salary</code>	<code>Double</code>	<code>DOUBLE</code>	Monthly or annual compensation (e.g., 50000.0).

32 Database Design (The Table Schema)

Our database has one table called `employees`. Here is how it is structured:

TABLE: EMPLOYEES			
Column	Type	Key / Notes	
<code>id</code>	<code>BIGINT</code>	<code>PRIMARY KEY, AUTO_INCREMENT</code>	
<code>name</code>	<code>VARCHAR(255)</code>	Full Name	
<code>email</code>	<code>VARCHAR(255)</code>	Email Address	
<code>department</code>	<code>VARCHAR(255)</code>	Department Name	
<code>salary</code>	<code>DOUBLE</code>	Numeric Salary Amount	

33 JPA Annotations (What Do Those @ Symbols Mean?)

In our Java code, you will see several annotations starting with `@`. Here is what each one does:

- `@Entity`: Tells Spring and Hibernate: "This Java class represents a table in the database."
- `@Table(name = "employees")`: Tells Hibernate to name the database table `employees`.
- `@Id`: Marks the `id` variable as the Primary Key of the table.
- `@GeneratedValue(strategy = GenerationType.IDENTITY)`: Tells the database to automatically increment the ID (1, 2, 3...) when a new employee is added.
- `@Repository`: Marks `EmployeeRepository` as a database access interface.
- `@Service`: Marks `EmployeeService` as a business logic service component.

- `@RestController`: Tells Spring: *"This class receives web requests and sends back data as JSON."*
- `@CrossOrigin(origins = "*")`: Allows web browsers running on any address to call our APIs without being blocked by CORS security.

34 REST API Design (The 6 Endpoints)

Our backend provides 6 standard REST API endpoints:

Method	Endpoint URL	What It Does	HTTP Status
POST	<code>/employees</code>	Creates and saves a new employee	201 Created
GET	<code>/employees</code>	Fetches all employees in a list	200 OK
GET	<code>/employees/{id}</code>	Fetches one employee by their ID number	200 OK / 404
PUT	<code>/employees/{id}</code>	Updates details of an existing employee	200 OK / 404
DELETE	<code>/employees/{id}</code>	Deletes an employee by ID	200 OK / 404
GET	<code>/employees/department/{dept}</code>	Returns all employees in a specific department	200 OK

35 API Request & Response Examples

1. Add Employee (POST /employees):

```
// Send this JSON to /employees:
{
  "name": "Rahul",
  "email": "rahul@gmail.com",
  "department": "IT",
  "salary": 50000.0
}

// Server returns (201 Created):
{
  "id": 1,
  "name": "Rahul",
  "email": "rahul@gmail.com",
  "department": "IT",
  "salary": 50000.0
}
```

2. View All (GET /employees):

```
// Send GET to /employees:
// Server returns (200 OK):
[
  {
    "id": 1,
    "name": "Rahul",
    "email": "rahul@gmail.com",
    "department": "IT",
    "salary": 50000.0
  },
  {
    "id": 2,
    "name": "Priya",
    "email": "priya@gmail.com",
    "department": "HR",
    "salary": 60000.0
  }
]
```

3. Update Details (PUT /employees/1):

4. Delete (DELETE /employees/1):

```
// Send this to /employees/1:
{
  "name": "Rahul Sharma",
  "email": "rahul.sharma@gmail.com",
  "department": "IT",
  "salary": 65000.0
}

// Server returns (200 OK):
{
  "id": 1,
  "name": "Rahul Sharma",
  "email": "rahul.sharma@gmail.com",
  "department": "IT",
  "salary": 65000.0
}
```

```
// Send DELETE to /employees/1:
// Server returns (200 OK):
{
  "message": "Employee deleted successfully with ID: 1"
}

// If ID does not exist (404 Not Found):
{
  "message": "Employee not found with ID: 99"
}
```

36 HTTP Status Codes (Quick Revision)

- **200 OK** : Success! Used when fetching (GET), updating (PUT), or deleting (DELETE) works properly.
- **201 Created** : Used specifically when a new employee is created and added to the database.
- **404 Not Found** : Used when you ask for an employee ID that does not exist in the database (like searching ID 999).
- **500 Internal Server Error** : Means the server crashed (e.g., database connection was lost).

37 How Department Filtering Works

In `EmployeeRepository.java`, we wrote one simple line:

```
List<Employee> findByDepartmentIgnoreCase(String department);
```

How does this work without writing SQL?

- Spring Data JPA reads the method name. It sees `findBy` + `Department` + `IgnoreCase`.
- It automatically creates this SQL query behind the scenes: `SELECT * FROM employees WHERE UPPER(department) = UPPER('IT');`
- The `IgnoreCase` part means it doesn't matter if you type "it", "IT", or "It"—it will find all employees in that department!

38 How Search Works (Client-Side Search)

When you type in the search bar on the webpage, how does it find employees so fast?

- When the webpage first loads, JavaScript downloads all employees and keeps them in an array called `allEmployees` in memory.
- When you type in `#searchInput`, JavaScript's `filterAndRender()` checks the query against 4 fields:

```
emp.name.toLowerCase().includes(query) ||
emp.email.toLowerCase().includes(query) ||
emp.department.toLowerCase().includes(query) ||
emp.id.toString().includes(query)
```

- Because this filtering happens right inside your browser's memory, the search results update instantly with zero delay!

39 How Dashboard Statistics Are Calculated

The 4 cards at the top of the webpage are calculated by `updateStats()` in `app.js`:

1. **Total Employees:** `employees.length` (e.g., 5).
2. **Active Departments:** Uses a JavaScript `Set` to count only unique departments: `new Set(employees.map(e => e.department)).size`.
3. **Total Payroll:** Uses JavaScript `.reduce()` to add up all salaries: `employees.reduce((sum, e) => sum + (e.salary || 0), 0)`.
4. **Average Salary:** Divides total payroll by total employee count: `Math.round(totalPayroll / totalCount)`.

40 Testing Architecture (How We Test)

We test our application using **Spring Boot Test** and **MockMvc** in `EmployeeControllerTest.java`.

- Before each test runs, `employeeRepository.deleteAll()` clears the database so every test starts fresh and clean.
- The tests run against an isolated in-memory test database (`jdbc:h2:mem:testdb`) defined in `application-test.properties`. This ensures testing never touches or deletes real data.

41 All 6 Automated Test Cases Explained

ID	Test Method	What It Tests	Result
TC-01	<code>testAddEmployee()</code>	Sends a POST request to add "Rahul" (IT, 50000). Verifies status 201 and non-null ID.	PASSED
TC-02	<code>testGetAllEmployees()</code>	Saves 2 employees and sends GET <code>/employees</code> . Verifies list has size 2.	PASSED
TC-03	<code>testGetEmployeeById()</code>	Saves an employee, gets it by ID. Verifies the returned name matches.	PASSED
TC-04	<code>testUpdateEmployee()</code>	Sends PUT <code>/employees/{id}</code> to change salary to 65000. Verifies updated salary.	PASSED
TC-05	<code>testGetEmployeesByDepartment()</code>	Saves 2 IT employees and 1 HR employee. Calls <code>/department/IT</code> . Verifies only the 2 IT employees come back.	PASSED

ID	Test Method	What It Tests	Result
TC-06	<code>testDeleteEmployee()</code>	Deletes an employee. Verifies delete success message, then checks that fetching that ID returns 404 Not Found.	PASSED

42 Why MockMvc?

Normally, to test a website, you have to start the server, open a browser, click buttons, and check if it worked. This is slow and tedious.

MockMvc allows Java to pretend it is a web browser. It sends fake HTTP requests directly to the controller inside memory. It tests URL paths, status codes, and JSON responses in less than 3 seconds!

43 Postman Testing Guide

The repository includes a ready-made Postman collection file:

`Employee_Management_API.postman_collection.json`.

How to use it:

1. Open Postman on your computer.
2. Click **Import** and select `Employee_Management_API.postman_collection.json`.
3. You will see all 6 requests pre-configured with headers and JSON bodies. Just click **Send**!

44 Error Handling (What Happens When an ID Doesn't Exist?)

In `EmployeeService.java`, when you try to update or delete an employee with an ID that doesn't exist:

```
.orElseThrow(() -> new RuntimeException("Employee not found with ID: " + id));
```

In `EmployeeController.java`, this error is caught with a `try-catch` block. The controller sends back an HTTP `404 Not Found` status with a friendly JSON message:

```
{ "message": "Employee not found with ID: 99" }
```

45 Security Analysis (Current State vs Future Production)

Security Feature	Current State in Project	What Should Be Added for Production
User Login	None (all endpoints are open).	Spring Security with login passwords and JWT tokens.
Permissions (RBAC)	Anyone can delete or edit employees.	Only users with <code>ROLE_ADMIN</code> can delete; others can only view.
CORS	<code>@CrossOrigin(origins = "*")</code> allows any website.	Restrict to the company's official domain name only.

Security Feature	Current State in Project	What Should Be Added for Production
Database Password	Written in <code>application.properties</code> .	Hide passwords using environment variables.

46 Dockerfile Explained (Line by Line)

A Dockerfile is a recipe that builds a container image for our project. Here is our `Dockerfile`:

```
# Stage 1: Build the JAR file
FROM maven:3.9.9-eclipse-temurin-17-alpine AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY src src
RUN mvn clean package -DskipTests

# Stage 2: Create the small runtime image
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/employee-management-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar", "--spring.profiles.active=h2"]
```

47 Why Use a Multi-Stage Docker Build?

STAGE 1 VS STAGE 2 IN DOCKER

```
[ Stage 1: Heavy Build Image (~850 MB) ]
Has Java Compiler, Maven, Source Code → Compiles app.jar → Discarded!
↓ Only app.jar is copied over
[ Stage 2: Clean Lightweight JRE Image (~180 MB) ]
Has ONLY the lightweight JRE and app.jar → Deployed to Cloud!
```

- **Saves 75% Disk Space:** Our final container image is only around 180 MB instead of almost 1 GB!
- **More Secure:** The final container does not have compilers, build tools, or source code inside it. Attackers cannot compile anything if they break in.

48 Docker Compose (Running App + MySQL Together)

`docker-compose.yml` lets you start both the Spring Boot app and a MySQL database with a single command:

```
docker compose up -d .
```

- **Service 1 (mysqldb):** Runs official MySQL 8.0 on port 3306. Saves database files permanently inside a volume named `db_data`.
- **Service 2 (app):** Builds our Spring Boot app from the Dockerfile, exposes port 8080, and connects to the MySQL container over an internal Docker network.

49 Cloud Deployment on Render

Our project is deployed live on the internet using **Render** at:

<https://employee-management-system-2ne1.onrender.com/>

How the deployment works:

1. Whenever we push new code to GitHub, Render detects the push.
2. Render reads our `Dockerfile` and builds the Docker container image.
3. Render starts the container and maps port 8080 to a public HTTPS web address.

50 Why Render? (And not AWS or Azure?)

- **Simple & Free:** AWS ECS and Kubernetes require complex networking, security groups, and cost money every month. Render gives us free container hosting without credit card charges.
- **Automatic HTTPS:** Render automatically gives us an SSL certificate (padlock icon) for free.
- **GitHub Integration:** Git push → automatic deployment!

51 Step-by-Step Guide: How to Run on Your Laptop

OPTION A: THE FASTEST WAY (ZERO INSTALLATION WITH H2 DATABASE)

Open PowerShell / Terminal in the project folder and type:

Windows: `.\mvnw.cmd spring-boot:run "-Dspring-boot.run.profiles=h2"`

Mac / Linux: `./mvnw spring-boot:run -Dspring-boot.run.profiles=h2`

Open your browser and visit: <http://localhost:8080/>

You can also view the H2 Database Console at: <http://localhost:8080/h2-console>

OPTION B: RUN WITH MYSQL AND DOCKER

If you have Docker Desktop installed, just run:

`docker compose up -d`

Both the MySQL database and the Spring Boot application will start automatically.

OPTION C: RUN AUTOMATED TESTS

To run all 6 integration tests, type:

`.\mvnw.cmd test` (Windows) or `./mvnw test` (Mac/Linux)

52 Configuration Files Explained

- `application.properties`: Used when running with MySQL. Contains MySQL URL (`localhost:3306/employee_db`), username, and password.
- `application-h2.properties`: Used when running with H2. Uses memory database URL (`jdbc:h2:mem:employee_db`) and enables the web console.
- `application-test.properties`: Used only when running automated tests (`jdbc:h2:mem:testdb`) so tests don't touch development data.

53 Project Folder Structure Tree

```

Employment_Management_System/
├── src/
│   ├── main/
│   │   ├── java/com/management/employee/
│   │   │   ├── EmployeeManagementApplication.java # Main class with main() method
│   │   │   ├── controller/EmployeeController.java # REST API endpoints (/employees)
│   │   │   ├── entity/Employee.java # Database table entity blueprint
│   │   │   ├── repository/EmployeeRepository.java # Database CRUD interface
│   │   │   └── service/EmployeeService.java # Business logic operations
│   │   └── resources/
│   │       ├── application.properties # MySQL settings
│   │       ├── application-h2.properties # H2 memory database settings
│   │       └── static/
│   │           ├── index.html # Webpage HTML
│   │           ├── styles.css # Webpage styles & themes
│   │           └── app.js # JavaScript frontend logic
│   └── test/
│       ├── java/com/management/employee/
│       │   └── EmployeeControllerTest.java # 6 automated MockMvc tests
│       └── resources/
│           └── application-test.properties # Test database settings
├── Dockerfile # Docker container blueprint
├── docker-compose.yml # Multi-container setup (App + MySQL)
├── Employee_Management_API.postman_collection.json # Postman tests file
├── pom.xml # Maven dependencies and versions
└── README.md # Documentation file

```

54 Code Quality Review (Good Points & Future Fixes)

Good Things in the Code:

- Constructor injection used everywhere (safe and easy to test).
- Clean naming for all variables and functions.
- HTML escaping in JavaScript (`escapeHtml()`) prevents XSS injection bugs.
- Clean separation: Controller does HTTP, Service does logic, Repository does database.

Things That Can Be Improved:

- Add a DTO layer (Data Transfer Objects) instead of exposing the database Entity directly.
- Use `@ControllerAdvice` for global error handling instead of try-catch blocks.
- Add `@NotBlank` and `@Positive` annotations for backend input validation.

55 Design Patterns Used in the Project

- **Layered Pattern:** Splitting the app into Controller → Service → Repository → Database.
- **Repository Pattern:** Using `EmployeeRepository` interface to hide database details from the rest of the application.
- **Dependency Injection (DI / IoC):** Spring Boot creates objects (beans) and injects them into constructors automatically.

- **Singleton Pattern:** Spring Boot creates only one instance of `EmployeeService` and `EmployeeRepository` and shares it across the app.

56 Performance Review

- **Query Speed:** H2 database queries finish in under 5 milliseconds.
- **Search Speed:** Searching in the browser table is instant (under 10 milliseconds) because JavaScript searches in memory.
- **Transparency:** Note that no formal multi-user stress testing (like 10,000 users at once) was done on this project.

57 Scalability Plan (What If Employee Count Grows?)

- **100 to 1,000 Employees:** The current project works smoothly without any changes.
- **10,000 Employees:** Sending 10,000 employees in one JSON list will be too slow. We should add **pagination** (showing 50 employees per page: `?page=0&size=50`).
- **100,000+ Employees:** We should add an index to the `department` column in MySQL (`CREATE INDEX idx_dept ON employees(department)`) and use Redis cache for frequently searched data.

58 Current Limitations (Being Completely Honest)

- No login screen or password protection (anyone who visits the site can view/delete).
- No pagination (all records are loaded in one big list).
- Only 1 database table (no separate table for departments or managers).
- Hard delete (when you delete an employee, the row is permanently gone—no recycle bin).

59 Future Enhancements (What to Build Next)

- **Spring Security + JWT:** Add login screen and assign roles (Admin vs Regular User).
- **Attendance & Leave:** Track daily attendance and allow employees to request time off.
- **Export to PDF / Excel:** Add a button to download the employee table or salary slips.
- **Soft Delete:** Keep deleted records marked as `is_deleted = true` so history is preserved.

60 Why These Features? (Managing Scope)

Why did we build employee CRUD, department filtering, and statistics instead of building attendance or payroll first?

- **First Things First:** You cannot have attendance or payroll without a working employee list first! Employee management is the core foundation.
- **Keep It Reliable:** Instead of building 10 half-working features, we built 6 solid features with 100% working automated tests, Docker containerization, and live cloud deployment.

61 End-to-End Walkthrough 1: Adding a New Employee

Let's follow the entire story of adding an employee from start to finish:

SCENARIO: ADDING "ANANYA VERMA" (ENGINEERING, ₹75,000)

1. **You click "Add Employee":** A popup dialog opens on your screen.
2. **You type in details:** Name = Ananya Verma, Email = ananya@gmail.com, Dept = Engineering, Salary = 75000.
3. **You click "Save":** JavaScript reads your inputs, formats them into JSON, and sends an HTTP POST to `/employees`.
4. **Controller catches it:** `EmployeeController` receives the request and turns the JSON into a Java `Employee` object.
5. **Service and Repository save it:** `EmployeeService` calls `EmployeeRepository.save()`.
6. **Database saves row:** Hibernate writes an `INSERT INTO employees...` SQL query. The database gives Ananya a new ID (e.g., ID 3).
7. **Server replies:** The controller sends back `201 Created` with the new employee JSON.
8. **Webpage updates:** The popup box closes, a green toast notification says *"Employee saved successfully!"*, and Ananya appears in the table. The total payroll and employee count cards increase automatically!

62 End-to-End Walkthrough 2: Filter by Department

SCENARIO: FILTERING BY "IT" DEPARTMENT

1. **You pick "IT" from dropdown:** In the control bar, you select "IT".
2. **JavaScript makes a call:** JavaScript executes `fetch('/employees/department/IT')`.
3. **Controller delegates:** `EmployeeController` calls `employeeService.getEmployeesByDepartment("IT")`.
4. **Repository derived query:** `findByDepartmentIgnoreCase("IT")` runs: `SELECT * FROM employees WHERE UPPER(department) = 'IT';`
5. **Results displayed:** Only IT department employees appear in the table.

63 End-to-End Walkthrough 3: Update Employee Details

SCENARIO: CHANGING SALARY FROM 50000 TO 65000

1. **Click Edit:** You click the pen icon next to Rahul (ID 1).
2. **Popup opens:** The form opens with Rahul's current details pre-filled.
3. **Change Salary:** You change the salary to 65000 and click Save.
4. **PUT request sent:** JavaScript sends an HTTP `PUT /employees/1` with the new details.
5. **Database updated:** `EmployeeService` finds employee 1, updates the fields, and saves to database.
6. **Confirmation:** Server returns `200 OK`, the table updates, and the stats card updates the average salary.

64 Master System Data Flow Diagram

COMPLETE MASTER DATA FLOW

```
[ USER / Web Browser ]
↓ ↑ (HTTP / JSON via fetch API)
[ Spring Boot Controller: EmployeeController ]
↓ ↑ (Method calls & returns)
[ Business Logic Service: EmployeeService ]
↓ ↑ (JpaRepository proxy)
[ Persistence: EmployeeRepository + Hibernate ]
↓ ↑ (SQL INSERT / SELECT / UPDATE / DELETE)
[ Database: MySQL 8.0 or In-Memory H2 ]
```

65 Technology Decision Matrix (Quick Table)

What We Chose	Alternative Option	Why We Chose It (Simple Explanation)
Java 17	Python / Node.js	Strong typing prevents salary and ID bugs; enterprise standard for backend software.
Spring Boot	Jakarta EE / Express	Tomcat is built-in; starters make REST APIs super fast to build.
Spring Data JPA	Plain JDBC	Eliminates 90% of repetitive SQL boilerplate code.
Hibernate	Direct SQL	Automatically updates database tables and maps Java objects to rows.
MySQL 8.0	MongoDB	Employee records fit perfectly into structured tables with ACID guarantees.
H2 Database	Local MySQL only	Allows anyone to run the project in seconds without installing MySQL.
Vanilla JS	React / Angular	No npm install or Node.js needed; frontend is bundled inside the Java JAR file.
Docker	Manual deploy	Packages the app into a container that runs identically on any machine or cloud server.

66 "Why This and Not That" (Common Viva Comparisons)

1. Spring Data JPA vs Plain JDBC:

Plain JDBC requires writing 30 lines of code just to read one employee by ID. Spring Data JPA gives you `findById()` in a single line. It saves time and prevents SQL typos.

2. Vanilla JavaScript vs React:

React is great for huge apps like Facebook, but for an employee dashboard it introduces 300MB of dependencies and complex build scripts. Plain HTML/CSS/JS is fast, lightweight, and packs right inside the Spring Boot JAR file.

3. H2 vs MySQL for Testing:

Testing against MySQL requires setting up passwords and cleaning up rows afterwards. H2 runs in memory, runs tests in 3 seconds, and deletes everything automatically when tests finish.

67 Versions Used & Compatibility

- **Java 17 LTS:** The stable baseline. Compatible with newer Java 21 and Java 23 runtimes.
- **Spring Boot 3.3.3:** The latest stable branch of Spring Boot 3. Requires Java 17+.
- **MySQL 8.0:** The modern MySQL release with high performance and strong security.

68 Requirements Traceability Table

Requirement	Java File	Automated Test	Status
FR-01 Add Employee	<code>EmployeeController.java</code> (line 27)	<code>testAddEmployee()</code>	VERIFIED
FR-02 View All	<code>EmployeeController.java</code> (line 34)	<code>testGetAllEmployees()</code>	VERIFIED
FR-03 Lookup by ID	<code>EmployeeController.java</code> (line 41)	<code>testGetEmployeeById()</code>	VERIFIED
FR-04 Update	<code>EmployeeController.java</code> (line 49)	<code>testUpdateEmployee()</code>	VERIFIED
FR-05 Delete	<code>EmployeeController.java</code> (line 61)	<code>testDeleteEmployee()</code>	VERIFIED
FR-06 Dept Filter	<code>EmployeeController.java</code> (line 73)	<code>testGetEmployeesByDepartment()</code>	VERIFIED

69 Test Coverage Disclosure

Our project contains **6 automated integration tests** in `EmployeeControllerTest.java`. Each test verifies a specific HTTP endpoint and asserts that the HTTP status code (200 or 201) and JSON data match expectations. All 6 tests pass successfully.

70 Deployment Traceability

How our code goes from GitHub to the Cloud:

GitHub Repository → Dockerfile → Docker Container Image → Render Cloud → Live Website

71 Project Maturity Review

- **What is Ready:** Full CRUD operations, department search, live dashboard stats, dual database support, automated tests, Docker containerization, and live cloud deployment.
- **What is Missing for Big Enterprise Use:** User login passwords (authentication), role permissions, pagination, and multi-table relationships.

72 Technical Interview Q&A (Memorize These!)

Q1: What does `@RestController` do in Spring Boot?

A: It is a combination of `@Controller` and `@ResponseBody`. It tells Spring that this class handles web requests and converts the returned Java objects directly into JSON.

Q2: What is the difference between `@GetMapping` and `@PostMapping`?

A: `@GetMapping` is used to fetch/read data without changing the server state. `@PostMapping` is used to send new data in the request body to create a new record in the database.

Q3: How does Spring Data JPA create queries from method names?

A: It reads method name keywords like `findByDepartmentIgnoreCase`. It parses `findBy` (SELECT), `Department` (the column), and `IgnoreCase` (UPPER() comparison) and generates the SQL automatically.

Q4: Why do we use Constructor Injection instead of `@Autowired` on fields?

A: Constructor injection makes variables `final` (immutable), guarantees all required dependencies are provided, and makes writing unit tests much easier without needing reflection.

73 Viva / Project Defense Questions

Q1: Why did you use two databases (MySQL and H2)?

A: MySQL is for permanent, real-world storage. H2 runs in memory with zero installation, which allows teachers, reviewers, or automated test runners to execute the project instantly without setting up a MySQL server.

Q2: What happens if I try to delete an employee ID that doesn't exist?

A: `EmployeeService` checks `existsById(id)`. If false, it throws a `RuntimeException`. The controller catches this and returns HTTP status `404 Not Found` with JSON `{"message": "Employee not found with ID: X"}`.

Q3: Why did you not use React for the frontend?

A: Using plain HTML, CSS, and JavaScript allows us to package the frontend right inside the Spring Boot JAR file. The entire app (frontend + backend) runs as a single self-contained JAR file under 50 MB with zero Node.js dependencies.

74 Final Project Summary

Problem: Managing employee records in Excel spreadsheets leads to duplicate IDs, accidental overwrites, and no REST API integration.

Solution: We built the Employment Management System—a full-stack web application using Java 17, Spring Boot 3.3.3, Spring Data JPA, MySQL/H2, and a responsive web dashboard.

Outcome: Complete CRUD functionality, real-time department filtering, live statistics, automated MockMvc tests, multi-stage Docker builds, and live deployment on Render.

75 Appendix: Quick Commands & SQL Cheat Sheet

1. Useful Terminal Commands:

```
# Run with H2 In-Memory Database (No setup needed):  
.\mvnw.cmd spring-boot:run "-Dspring-boot.run.profiles=h2"    (Windows)  
./mvnw spring-boot:run -Dspring-boot.run.profiles=h2          (Mac/Linux)  
  
# Run with MySQL:  
./mvnw spring-boot:run  
  
# Run all 6 automated tests:  
./mvnw test  
  
# Start with Docker Compose (MySQL + App):  
docker compose up -d
```

2. MySQL Table DDL:

```
CREATE TABLE employees (  
    id BIGINT NOT NULL AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(255),  
    email VARCHAR(255),  
    department VARCHAR(255),  
    salary DOUBLE  
);
```

3. Key Terminology:

- **CRUD:** Create, Read, Update, Delete.
- **REST:** Representational State Transfer (architectural style for web APIs).
- **ORM:** Object-Relational Mapping (translating Java objects to database rows).
- **MockMvc:** Testing library that simulates HTTP web requests in memory.
- **LTS:** Long-Term Support (software version supported for multiple years).